

Phát triển kỹ thuật

RESTful API, SWAGGER, MVC, MSA

TS. Trần Đức Minh



LESSON OVERVIEW

- REST, API và RESTful API
- HTTP Methods
- HTTP Status Codes
- Kiến trúc MVC và MSA
- Tài liệu API, chuẩn OAS và Swagger

REST LÀ GÌ ?

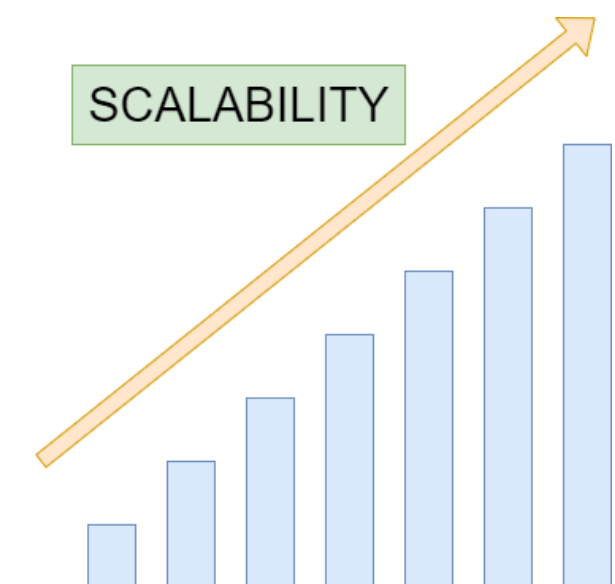
- **REST (Representational State Transfer)** là phong cách kiến trúc phần mềm do Roy Fielding đề xuất năm 2000. REST không phải là giao thức, REST là tập hợp các ràng buộc thiết kế giúp hệ thống web trở nên ***nhất quán, có thể mở rộng và dễ tích hợp***.

- ***Tính nhất quán:***

- Ví dụ không tuân theo REST (không nhất quán)
 - Xem thông tin nhân viên số 5: GET /viewEmployee?id=5
 - Sửa thông tin nhân viên số 5: POST /update_emp_v2/5
- Ví dụ tuân theo REST (nhất quán)
 - Xem thông tin nhân viên số 5: GET /employees/5
 - Sửa thông tin nhân viên số 5: PUT /employees/5

REST LÀ GÌ ?

- ***Tính có thể mở rộng*** (Scalability) là khả năng hệ thống vẫn vận hành mượt mà, không bị quá tải khi số lượng người dùng hoặc lượng dữ liệu tăng lên gấp 10, gấp 100 lần.
 - *Stateful* (Kém mở rộng): Server lưu trữ lịch sử giao dịch, khi thêm Server mới sẽ bị thiếu dữ liệu, gây nghẽn hệ thống do phải đồng bộ liên tục.
 - *Stateless* (REST): Request chứa đầy đủ 100% thông tin, mọi Server đều có thể xử lý độc lập mà không cần biết lịch sử trước đó.
 - *Kết quả*: Dễ dàng nhân bản hàng trăm Server để đáp ứng hàng triệu truy cập cùng lúc mà không lo sập hệ thống.



REST LÀ GÌ ?

- **Tính dễ tích hợp** (Interoperability) là khả năng các hệ thống công nghệ hoàn toàn khác nhau (khác về ngôn ngữ lập trình, hệ điều hành, hay nền tảng phần cứng) vẫn có thể kết nối, hiểu và trao đổi dữ liệu với nhau.
- REST dễ tích hợp nhờ hai yếu tố cốt lõi: Sử dụng giao thức phổ quát (HTTP) và Định dạng dữ liệu tiêu chuẩn (thường là JSON hoặc XML).
 - Trong một hệ thống RESTful, hai thành phần tham gia giao tiếp phân tách rõ ràng:
 - **Client** (phía yêu cầu dữ liệu): Chỉ quan tâm đến giao diện, trải nghiệm người dùng và cách hiển thị dữ liệu.
 - **Server** (phía cung cấp dữ liệu): Chỉ quan tâm đến xử lý logic, tính toán và lưu trữ cơ sở dữ liệu.
 - Client và Server giao tiếp với nhau thông qua một "vùng đệm trung gian" đã được chuẩn hóa. Server không cần biết Client chạy bằng gì và Client cũng chẳng quan tâm Server dùng công nghệ nào.

API LÀ GÌ ?

- **API (Application Programming Interface)** là thành phần giúp các phần mềm, ứng dụng hoặc hệ thống khác nhau có thể nói chuyện, hiểu và trao đổi dữ liệu với nhau.
- Nếu không có API, các ứng dụng sẽ giống như những ốc đảo, không thể lấy được thông tin từ thế giới bên ngoài.
- Các loại API phổ biến:
 - **Web API** (RESTful API): Kết nối ứng dụng (Web/Mobile) với Server qua Internet bằng giao thức HTTP (Ví dụ: Đăng nhập, xem thời tiết, đặt vé).
 - **Library/Framework API**: Các hàm có sẵn trong thư viện ngôn ngữ lập trình (Python, Java...) giúp lập trình viên dùng ngay không cần viết lại từ đầu (Ví dụ: Vẽ biểu đồ, tính toán).
 - **Operating System API**: Giúp phần mềm giao tiếp và ra lệnh cho phần cứng của hệ điều hành (iOS, Android, Windows...) (Ví dụ: Xin quyền mở Camera, định vị GPS).

RESTful API LÀ GÌ ?

- **RESTful API** là thuật ngữ được kết hợp từ hai khái niệm: **API** và **REST**.
- RESTful API là một bộ quy chuẩn giúp các ứng dụng (như điện thoại, máy tính) giao tiếp với máy chủ (Server) thông qua mạng Internet một cách nhất quán, tường minh và hiệu quả.
- Một hệ thống API tuân thủ đúng các ràng buộc của kiến trúc REST thì nó được gọi là **RESTful API**.

CÁC RÀNG BUỘC CỦA KIẾN TRÚC RESTful

- **Client - Server** (Kiến trúc Khách - Chủ)
- **Stateless** (Không lưu trạng thái)
- **Cacheable** (Khả năng lưu bộ nhớ đệm)
- **Uniform Interface** (Giao diện đồng nhất)
- **Layered System** (Hệ thống phân tầng)
- ***Code on Demand*** (Mã lệnh theo yêu cầu - **TÙY CHỌN**)

Nếu vi phạm dù chỉ một trong số các ràng buộc bắt buộc này, hệ thống đó không còn được gọi là RESTful nữa.

Client - Server

- Ràng buộc này yêu cầu sự tách biệt tuyệt đối về mặt trách nhiệm giữa giao diện người dùng (**Client**) và logic xử lý, lưu trữ dữ liệu (**Server**).
 - **Bản chất:** Client không cần biết dữ liệu được lưu trữ trong Database nào hay Server cấu hình ra sao. Ngược lại, Server cũng không quan tâm đến việc giao diện hiển thị thế nào, chạy trên nền tảng mobile iOS, Android hay Website.
 - **Giá trị mang lại:** Giúp hai phía phát triển và nâng cấp hoàn toàn độc lập. Toàn bộ giao diện phía Client có thể đập đi xây lại mà không cần sửa một dòng code nào ở phía Server.

Stateless

- Đây là ràng buộc quan trọng để tối ưu hiệu năng và là yếu tố cơ bản cho tính mở rộng của REST.
 - **Bản chất:** Server không lưu giữ bất kỳ ngữ cảnh hay lịch sử phiên làm việc (session) nào của Client. Mỗi Request (yêu cầu) được gửi từ Client lên Server phải độc lập và chứa đầy đủ 100% thông tin cần thiết để Server hiểu và xử lý ngay lập tức (ví dụ: phải đính kèm token xác thực, ID tài nguyên cần thao tác).
 - **Giá trị mang lại:** Giúp hệ thống có thể mở rộng cực kỳ tốt (Scalability). Khi lượng truy cập tăng vọt, ta có thể bổ sung thêm Server chạy song song. Các Request có thể nhảy vào bất kỳ Server nào cũng xử lý được ngay mà không cần tốn thời gian đồng bộ dữ liệu phiên làm việc giữa các Server.

Cacheable

- **Cacheable** là ràng buộc về khả năng lưu thông tin ở bộ nhớ đệm trên Client.
 - **Bản chất:** Mọi dữ liệu phản hồi (Response) từ Server trả về cho Client phải được định nghĩa rõ ràng là **Cho phép lưu nháp** (Cacheable) hay **Không cho phép** (Non-cacheable).
 - **Giá trị:** Nếu dữ liệu ít khi thay đổi (như danh mục sản phẩm, ảnh logo, thông tin cấu hình), Client có thể lưu lại ngay trên bộ nhớ cục bộ của máy. Ở những lần truy cập sau, Client lấy luôn dữ liệu này ra dùng chứ không cần gọi lên Server nữa, giúp tiết kiệm băng thông mạng và tăng tốc độ tải trang đáng kể.

Cacheable

- Đây là ràng buộc cốt lõi, đó là bắt buộc các thành phần giao tiếp trong hệ thống phải dùng chung một "ngôn ngữ chuẩn hóa" duy nhất. Roy Fielding chia ràng buộc này thành 4 thuộc tính nhỏ:
 1. **Định danh tài nguyên** (Identification of resources): Mọi đối tượng dữ liệu phải được định danh bằng một đường dẫn duy nhất và phải là Danh từ (**URI**) (Ví dụ: /users/42, /products/7).
 2. **Thao tác với tài nguyên định dạng dữ liệu** (Manipulation of resources through representations): Client không tương tác trực tiếp với Database. Khi Client nắm giữ một định dạng dữ liệu của tài nguyên (thường là định dạng JSON hoặc XML) kèm theo quyền hạn, Client có thể gửi yêu cầu sửa (PUT) hoặc xóa (DELETE) tài nguyên đó trên Server.

Cacheable

- Đây là ràng buộc cốt lõi, đó là bắt buộc các thành phần giao tiếp trong hệ thống phải dùng chung một "ngôn ngữ chuẩn hóa" duy nhất. Roy Fielding chia ràng buộc này thành 4 thuộc tính nhỏ:
 3. **Thông điệp tự mô tả** (Self-describing messages): Mỗi thông điệp gửi đi phải tự giải thích rõ nó thuộc định dạng gì (ví dụ trong HTTP Header phải ghi rõ Content-Type: application/json) để phía nhận biết cách giải mã dữ liệu.
 4. **HATEOAS** (Hypermedia As The Engine Of Application State): Sau khi Client gọi một API, Server sẽ trả về dữ liệu kèm theo các đường link động hướng dẫn các hành động tiếp theo mà Client có thể thực hiện (như nút "Xem chi tiết", "Hủy đơn", "Thanh toán"), giúp Client tự điều hướng ứng dụng một cách linh hoạt.

Layered System

- Đây là ràng về hệ thống phân tầng
 - **Bản chất:** Client không thể biết và không cần biết nó đang kết nối trực tiếp với Server gốc chứa dữ liệu hay đang đi qua các tầng công nghệ trung gian ở giữa.
 - **Giá trị mang lại:** Hệ thống có thể chèn thêm các tầng như Cân bằng tải (Load Balancer), Bảo mật (Firewall/API Gateway), ... ở giữa Client và Server để tăng tính bảo mật và khả năng chịu tải, mà phía giao diện người dùng hoàn toàn không bị ảnh hưởng hay phải cấu hình lại.



Code on Demand

- Đây là ràng buộc duy nhất trong số 6 ràng buộc không bắt buộc.
 - **Bản chất:** Server có thể mở rộng chức năng cho Client bằng cách gửi kèm các đoạn mã thực thi (như mã JavaScript) để Client chạy trực tiếp trên trình duyệt hoặc ứng dụng của Client.
 - **Giá trị mang lại:** Giúp giảm tải việc tính toán cho Server (ví dụ: Server gửi một đoạn code JavaScript về để trình duyệt tự kiểm tra tính hợp lệ của các ô nhập liệu trước khi người dùng bấm gửi form).
 - Nếu API không dùng tính năng này, hệ thống vẫn hoàn toàn đạt chuẩn RESTful.

HTTP Methods

- **Các phương thức HTTP (HTTP Methods)** là các "hành động" chuẩn hóa mà Client gửi lên Server nhằm báo cho Server biết phải thực hiện thao tác gì với một tài nguyên (Resource) cụ thể.
 - HTTP Method chính là hành động mà ta muốn thực hiện trên URL.
- Năm phương thức HTTP cốt lõi trong RESTful API:
 - **GET**
 - **POST**
 - **PUT**
 - **PATCH**
 - **DELETE**

GET & POST

- **GET** (Lấy dữ liệu)
 - Ý nghĩa: Client yêu cầu Server trả về thông tin của tài nguyên mà không làm thay đổi bất kỳ dữ liệu nào trên hệ thống.
 - Ví dụ:
 - GET /students/42 => Server trả về dữ liệu hồ sơ của sinh viên có ID là 42.
 - GET /students => Server trả về một danh sách sinh viên.
- **POST** (Tạo mới dữ liệu)
 - Ý nghĩa: Client gửi dữ liệu lên Server để yêu cầu tạo mới tài nguyên.
 - Ví dụ:
 - POST /students => Thêm một sinh viên mới vào cơ sở dữ liệu (CSDL)
 - Kèm theo phương thức này là các thông tin về sinh viên cần thêm vào CSDL dưới dạng JSON hoặc XML.

PUT & PATCH

- **PUT** (Thay thế toàn bộ tài nguyên)

- Ý nghĩa: Ghi đè toàn bộ dữ liệu cũ của tài nguyên bằng một tập dữ liệu mới hoàn chỉnh. Nếu thông tin dữ liệu gửi lên bị thiếu trường nào, trường đó trên Server sẽ bị xóa hoặc chuyển giá trị về null.
- Ví dụ:
 - PUT /students/42 => Cập nhật lại toàn bộ thông tin của sinh viên có ID là 42.
 - Kèm theo phương thức này là toàn bộ thông tin về sinh viên cần cập nhật dưới dạng JSON hoặc XML.

- **PATCH** (Chỉ sửa một phần tài nguyên)

- Ý nghĩa: Chỉ cập nhật một hoặc một vài trường cần thay đổi thông tin, các trường dữ liệu khác được giữ nguyên.
- Ví dụ:
 - PATCH /students/42 => Sửa một vài thông tin của sinh viên có ID là 42
 - Kèm theo phương thức này là các thông tin về sinh viên cần thay đổi dưới dạng JSON hoặc XML.

DELETE

- **DELETE** (Xóa dữ liệu)
 - Ý nghĩa: Client yêu cầu Server hủy bỏ và xóa hoàn toàn tài nguyên được chỉ định.
 - Ví dụ:
 - `DELETE /students/42` => Yêu cầu xóa dữ liệu của sinh viên có ID là 42.
- **HEAD**: Giống hệt GET nhưng Server chỉ trả về phần Header chứ không trả về nội dung dữ liệu (Body).
 - Ví dụ: Dùng để kiểm tra xem một file dung lượng lớn có tồn tại không hoặc nặng bao nhiêu MB trước khi quyết định tải về.
- **OPTIONS**: Client gửi câu hỏi để kiểm tra xem cấu hình bảo mật của Server cho phép URL đó được quyền thực hiện những Methods nào.
 - Ví dụ Server trả về: GET, POST, OPTIONS.

Ví dụ: Thiết kế endpoint cho hệ thống quản lý sinh viên

GET	/api/students	Lấy danh sách tất cả sinh viên
-----	---------------	--------------------------------

GET	/api/students/{id}	Lấy thông tin 1 sinh viên theo ID
-----	--------------------	-----------------------------------

POST	/api/students	Thêm sinh viên mới
------	---------------	--------------------

PUT	/api/students/{id}	Cập nhật toàn bộ thông tin sinh viên
-----	--------------------	--------------------------------------

PATCH	/api/students/{id}	Cập nhật một số trường (vd: điểm số)
-------	--------------------	--------------------------------------

DELETE	/api/students/{id}	Xóa sinh viên
--------	--------------------	---------------

GET	/api/students/{id}/courses	Lấy danh sách môn học của sinh viên
-----	----------------------------	-------------------------------------

HTTP Status Codes

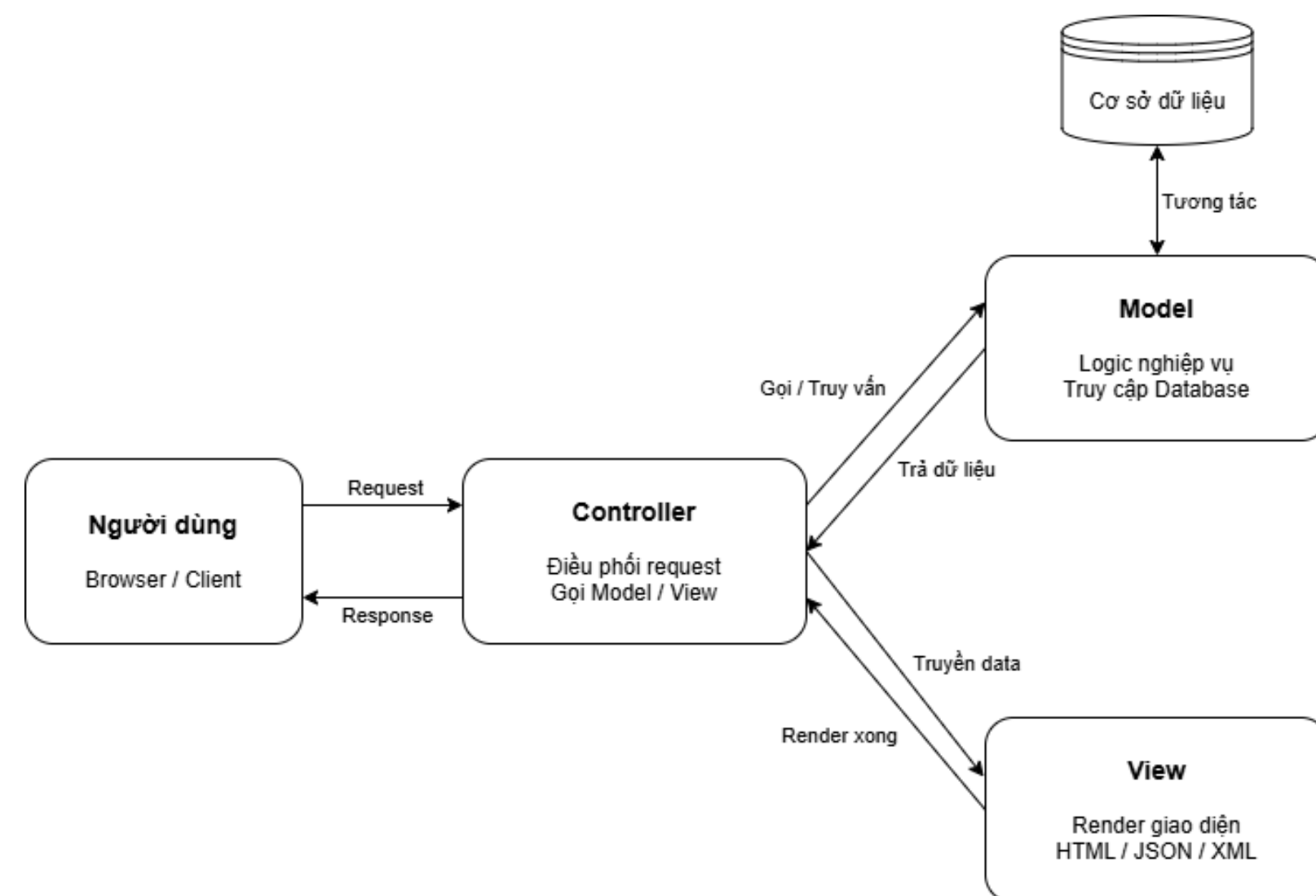
- **HTTP Status Code** (Mã trạng thái HTTP) là một chuỗi gồm 3 chữ số mà Server trả về cho Client sau khi nhận được một yêu cầu (Request).
- HTTP Status Code chính là lời phản hồi ngắn của Server báo cho Client biết rằng yêu cầu đó đã được xử lý thành công, thất bại hay gặp lỗi gì.
- Năm nhóm mã HTTP Status Codes:
 - **Nhóm 1xx**: Server thông báo đã nhận được yêu cầu và Client hãy tiếp tục đợi hoặc gửi tiếp dữ liệu.
 - **Nhóm 2xx**: Yêu cầu đã được Server tiếp nhận, hiểu và xử thành công.
 - **Nhóm 3xx**: Tài nguyên tìm đã bị chuyển sang một địa chỉ mới.
 - **Nhóm 4xx**: Yêu cầu không thể xử lý được vì lỗi xuất phát từ Client.
 - **Nhóm 5xx**: Yêu cầu hoàn toàn hợp lệ, nhưng Server bị lỗi, bị quá tải hoặc lập trình viên viết code bị lỗi thuật toán nên không xử lý được.

Một số HTTP Status Codes quan trọng

CODE	TÊN	Ý NGHĨA	DÙNG KHI
200	OK	Thành công, có dữ liệu trả về	GET, PUT, PATCH thành công
201	Created	Tạo resource thành công	POST thành công
204	No Content	Thành công, không có dữ liệu trả về	DELETE thành công
301	Moved Permanently	URL đã thay đổi vĩnh viễn	Redirect API version
400	Bad Request	Request không hợp lệ	Input sai format, thiếu trường
401	Unauthorized	Chưa xác thực	Thiếu hoặc sai token
403	Forbidden	Đã xác thực nhưng không có quyền	Không đủ role/permission
404	Not Found	Không tìm thấy resource	ID không tồn tại
422	Unprocessable Entity	Dữ liệu không hợp lệ về nghiệp vụ	Email trùng, validation fail
500	Internal Server Error	Lỗi server không xác định	Lỗi code / database crash
503	Service Unavailable	Server tạm thời không hoạt động	Bảo trì hoặc quá tải

Kiến trúc MVC hiện đại (Monolith)

- Trong các kiến trúc ứng dụng hiện đại, thành phần View không còn đảm nhận nhiệm vụ biên dịch và trả về toàn bộ giao diện HTML được dựng sẵn từ phía máy chủ (Server-Side Rendering) như trước.
- Thay vào đó, mô hình MVC hiện đại đã chuyển dịch sang đóng vai trò như một **API Provider**, tức là chỉ tập trung xử lý và cung cấp dữ liệu thô (thường là định dạng JSON/XML) để phía Client tự xử lý giao diện.

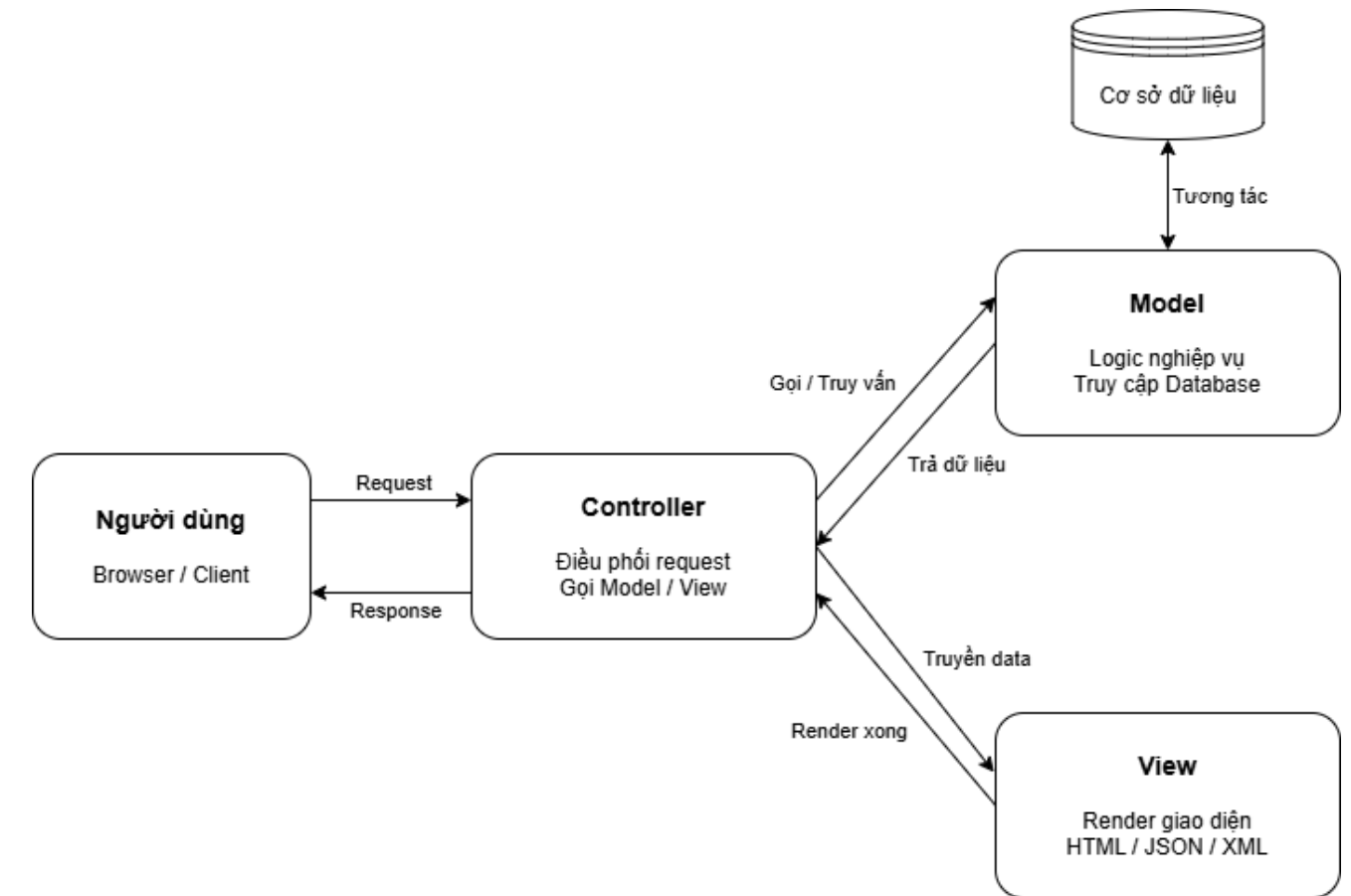


Vai trò các thành phần trong kiến trúc MVC hiện đại

- **Người dùng** (Browser/Client): Là nơi bắt đầu và kết thúc của một chu kỳ. Client gửi Request và nhận lại kết quả hiển thị (Response).
- **Controller** (Trung tâm điều phối): Controller không trực tiếp xử lý dữ liệu, cũng không tự vẽ giao diện. Nó chỉ làm nhiệm vụ gọi đúng người đúng việc: gọi Model khi cần dữ liệu và gọi View khi cần hiển thị.
- **Model** (Trọng tâm logic nghiệp vụ): Giữ vai trò xử lý các quy tắc nghiệp vụ (Business Logic) và là thành phần duy nhất kết nối với Database.
- **View** (Giao diện): Chỉ tập trung vào việc nhận dữ liệu từ Controller thô (raw data) và chuyển đổi (render) nó thành các định dạng mà Browser hiểu được (như HTML để hiện trang web, hoặc JSON/XML cho các ứng dụng Mobile App).

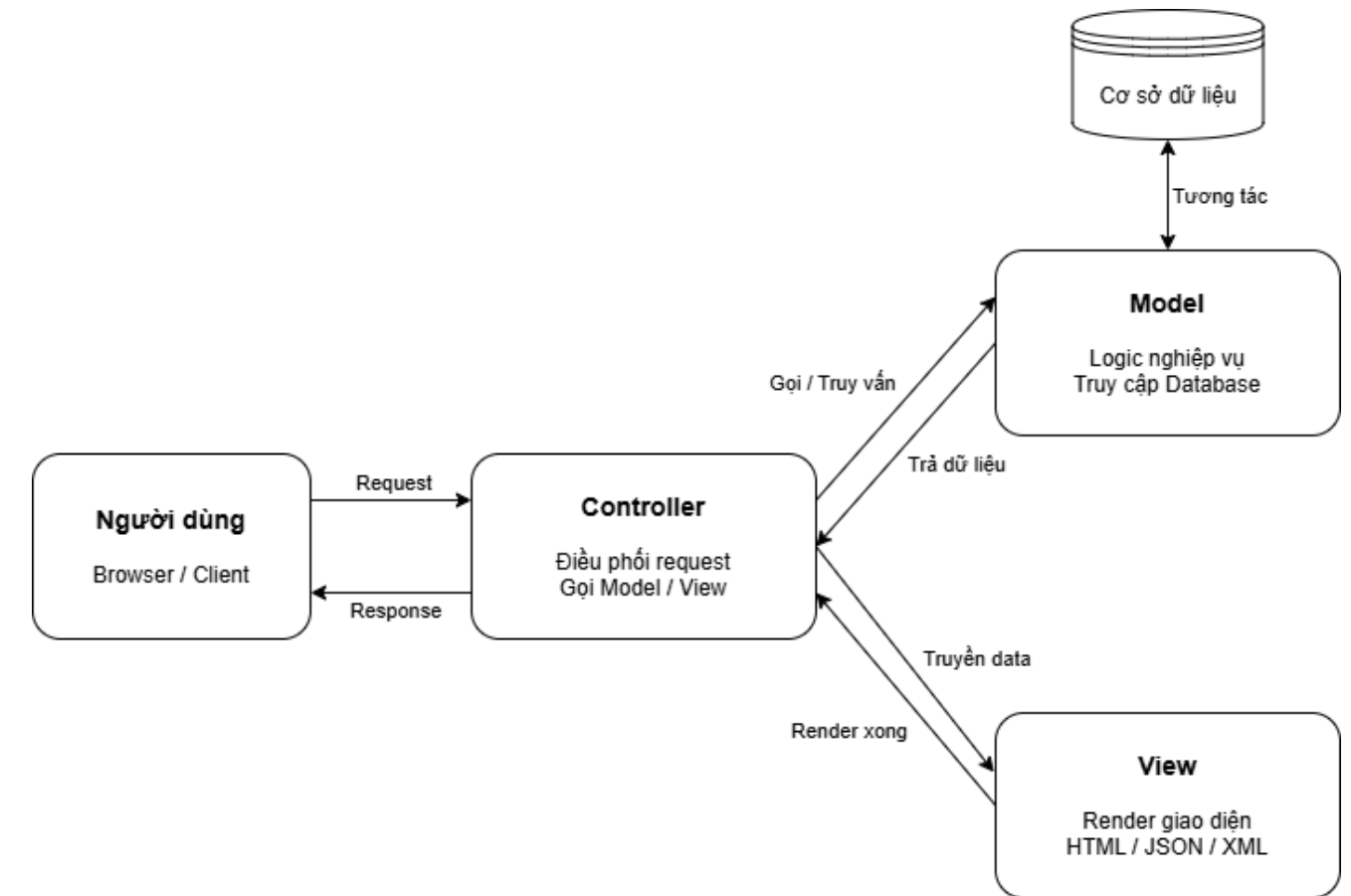
Luồng đi của dữ liệu (Data flow)

1. **Người dùng (Client) -> Controller (API Request):** Người dùng thực hiện một hành động (ví dụ: nhấn vào trang "Danh sách sản phẩm"). Ứng dụng Client sẽ gửi một Asynchronous HTTP Request với định dạng URL như **GET /api/products** đến Controller của Server.
2. **Controller -> Model** (Gọi nghiệp vụ): Controller tiếp nhận API request, kiểm tra các tham số (như phân trang, bộ lọc) và yêu cầu Model: "Hãy lấy cho tôi danh sách sản phẩm hợp lệ trong kho".
3. **Model -> Database -> Model:** Model thực hiện câu lệnh truy vấn cơ sở dữ liệu để lấy dữ liệu thô (Raw Data) lên và thực hiện các logic tính toán (nếu có).



Luồng đi của dữ liệu (Data flow)

4. **Model -> Controller** (Trả dữ liệu thô): Model trả lại danh sách dữ liệu dưới dạng các đối tượng lập trình (Entities/Objects) cho Controller.
5. **Controller -> View** (Truyền cấu trúc dữ liệu): Controller nhận dữ liệu từ Model, sau đó bàn giao dữ liệu này cho View và định hướng: "Hãy lọc bỏ các trường dữ liệu nhạy cảm (như giá vốn, mã định danh nội bộ) rồi đóng gói mảng dữ liệu này thành định dạng JSON chuẩn".



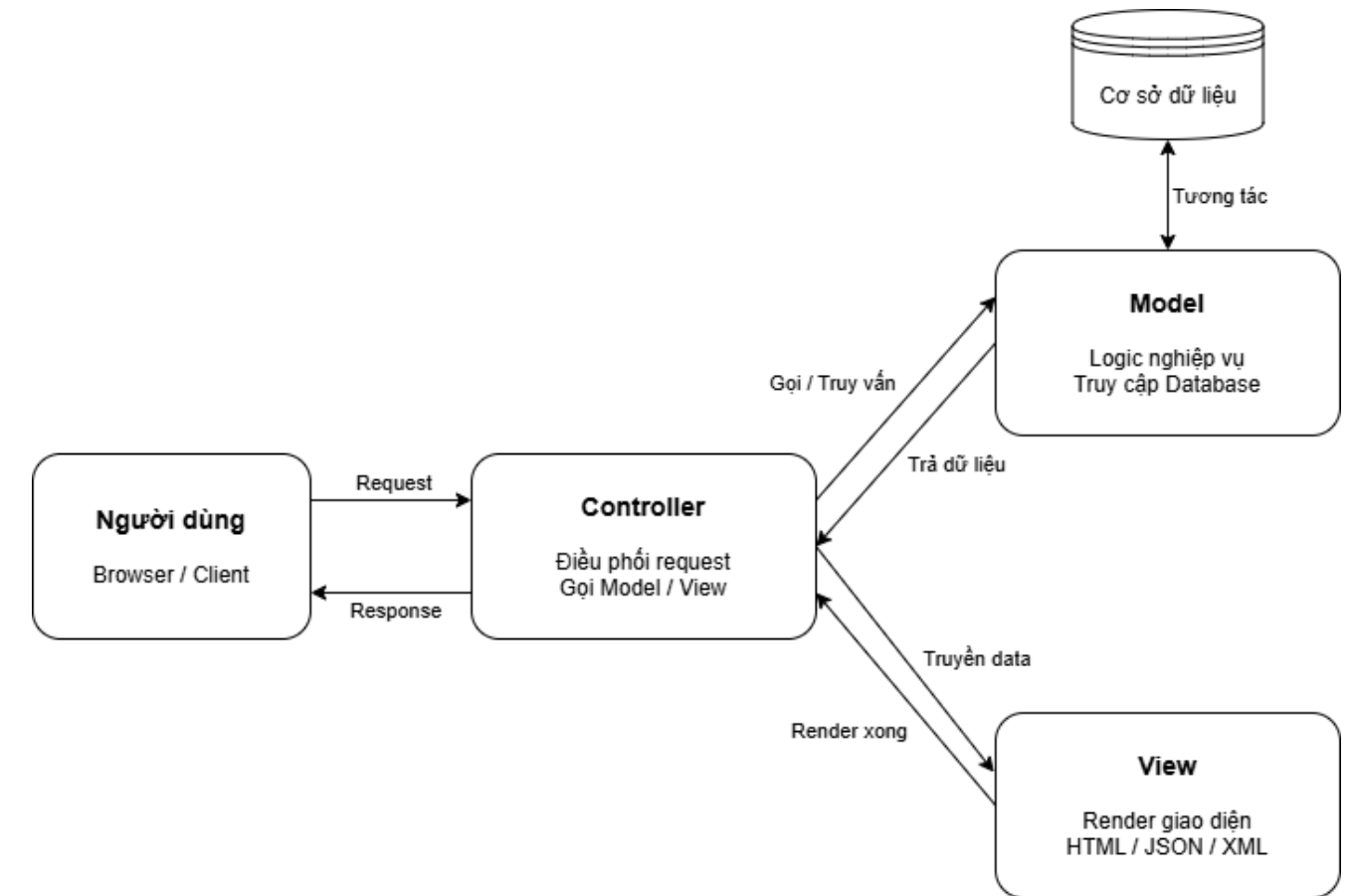
Luồng đi của dữ liệu (Data flow)

6. View (JSON Engine) -> Controller (Định dạng xong):

View tiến hành chuyển đổi mảng dữ liệu thành một chuỗi **JSON String** (cấu trúc gồm các cặp *key-value*) rồi trả lại cho Controller.

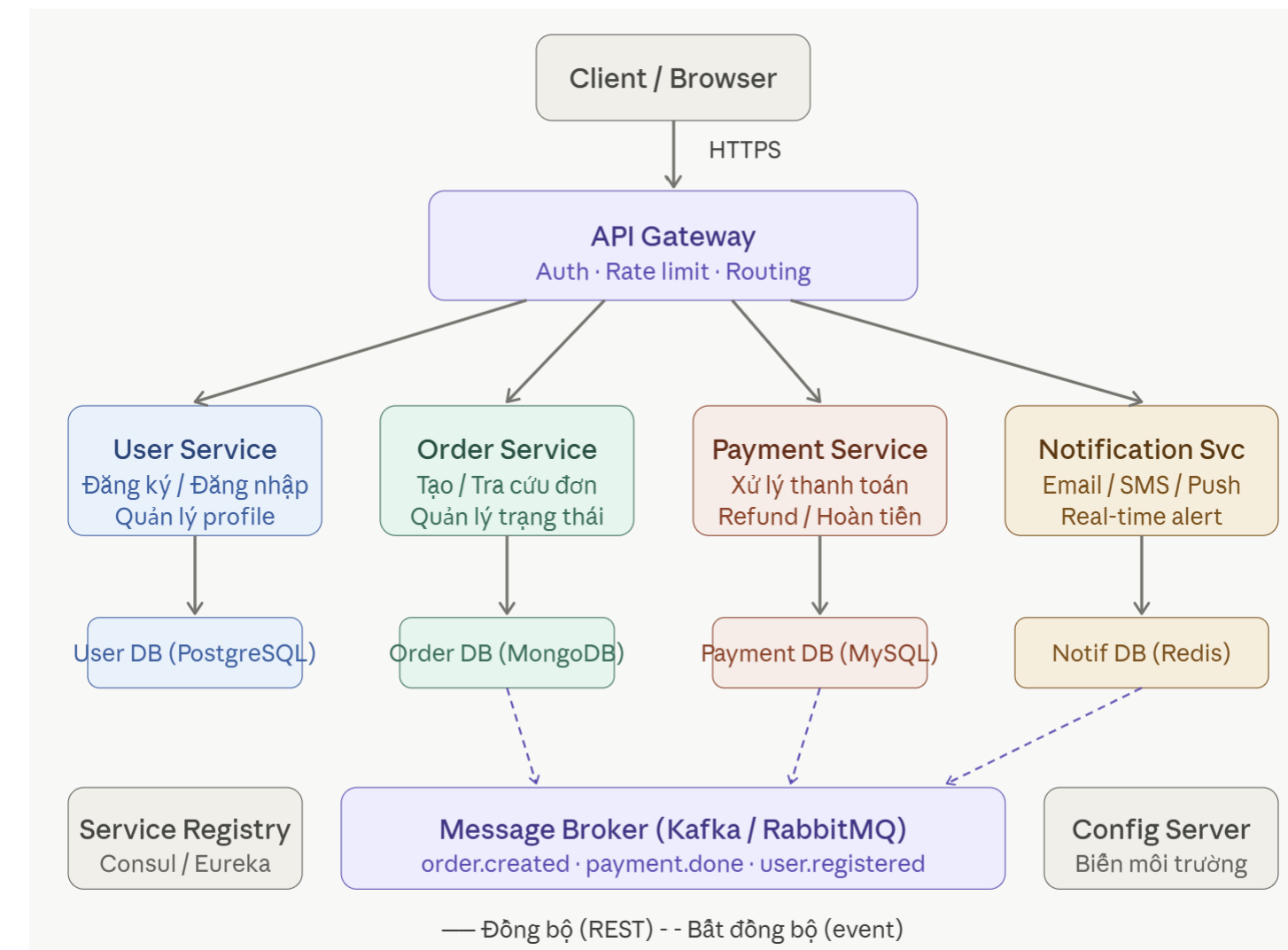
7. Controller -> Người dùng (API Response):

Controller thiết lập Header của phản hồi là *Content-Type: application/json* và gửi trả chuỗi JSON đó (HTTP Response) về cho Client. Client (Browser/App) nhận được chuỗi JSON này, bóc tách dữ liệu và tự động render lên màn hình cho người dùng thấy.



Kiến trúc MSA

- **MSA (Microservices Architecture** - Kiến trúc vi dịch vụ) là một phong cách thiết kế kiến trúc phần mềm hiện đại, trong đó một ứng dụng lớn sẽ được chia nhỏ thành một tập hợp các dịch vụ (service) độc lập, kích thước nhỏ và có thể kết nối với nhau.
- Các đặc điểm cốt lõi của MSA
 - Chia nhỏ theo nghiệp vụ
 - Cơ sở dữ liệu riêng biệt
 - Triển khai độc lập
 - Giao tiếp qua giao thức nhẹ



API Gateway

- Khi **Client** gửi một yêu cầu HTTPS đến **Server**, điểm chạm đầu tiên luôn là **API Gateway**. Trong MSA, các service phía sau thường nằm trong mạng nội bộ (Private Network) để bảo mật. API Gateway đóng vai trò là cổng phân phối duy nhất với 3 nhiệm vụ:
 - **Auth** (Xác thực/Phân quyền): Kiểm tra xem Token của User có hợp lệ hay không. Nếu hợp lệ, hệ thống mới cho đi tiếp, tránh việc từng service nhỏ phía sau phải tự viết lại code xác thực.
 - **Rate limit** (Giới hạn tải): Ngăn chặn tấn công DDoS hoặc spam API. Ví dụ: Thiết lập tối đa một IP chỉ được gọi API 100 lần/phút.
 - **Routing** (Định tuyến): Đọc URL để điều hướng. Ví dụ, request có tiền tố **/api/v1/orders** sẽ được Gateway chuyển thẳng về hướng Order Service.

Microservices và lưu trữ dữ liệu phi tập trung

- Đây là nơi xử lý các nghiệp vụ cốt lõi. Chọn hệ quản trị cơ sở dữ liệu phù hợp với đặc thù nghiệp vụ:
 - **User Service + PostgreSQL:** Quản lý tài khoản cần tính toán vẹn dữ liệu cực cao, ràng buộc chặt chẽ -> Dùng cơ sở dữ liệu quan hệ (RDBMS) như Postgres là tối ưu.
 - **Order Service + MongoDB:** Đơn hàng TMĐT thường có cấu trúc thay đổi tùy biến (ví dụ: đơn mua quần áo có size/màu, đơn mua đồ điện tử có số IMEI/thời hạn bảo hành). Dữ liệu dạng Document của NoSQL (MongoDB) giúp lưu trữ linh hoạt.
 - **Payment Service + MySQL:** Xử lý dòng tiền, hóa đơn, hoàn tiền -> Cần tính giao dịch tài chính chuẩn chỉ, an toàn tuyệt đối của RDBMS.
 - **Notification Svc + Redis:** Lưu các thông báo đẩy, hàng đợi tin nhắn tạm thời hoặc lịch sử alert theo thời gian thực -> Tốc độ đọc/ghi siêu tốc trên RAM của Redis là lựa chọn số một.

Giao tiếp hướng sự kiện (Message Broker)

- Trong MSA, nếu **Order Service** gọi trực tiếp (Đồng bộ - REST API) sang **Payment Service** và **Notification Service**, hệ thống sẽ bị chậm (vì phải đợi nhau xử lý) và dễ lỗi dây chuyền.
- Kiến trúc MSA đã giải quyết bằng Message Broker (Kafka / RabbitMQ) để giao tiếp Bất đồng bộ (Async):
 - Các dịch vụ chỉ việc gửi các **Events** như *order.created* (đơn hàng đã tạo) hay *payment.done* (thanh toán hoàn tất) lên trục trung tâm này rồi quay lại phục vụ tiếp các request khác.
 - Các dịch vụ khác quan tâm đến các **Events** đó sẽ tự động **Subscribe** (đăng ký lắng nghe) để nhận tin nhắn về xử lý theo tiến độ riêng của mình.

Bổ trợ

- **Service Registry:** Các dịch vụ trong MSA thường có thể tự động tắt đi, bật lại hoặc nhân bản lên nhiều phiên bản ở các địa chỉ IP khác nhau. Service Registry giống như tổng đài. Khi **Order Service** muốn gọi **Payment Service**, nó sẽ hỏi Registry: "Hiện tại Payment Service đang chạy ở IP nào?" để lấy địa chỉ chính xác.
- **Config Server:** Thay vì lưu các biến môi trường (Mật khẩu DB, Link API, Khóa bí mật) rải rác ở file .env của từng service, toàn bộ cấu hình được tập trung tại đây. Khi cần đổi mật khẩu Database của 10 service, bạn chỉ cần sửa 1 nơi duy nhất trên Config Server.

Luồng đi của dữ liệu trên MSA

Ví dụ: Hành trình đặt mua một món hàng. Khách hàng bấm nút "Xác nhận đặt hàng và Thanh toán tiền mặt (COD)" trên App di động.

1. **Gửi request:** App gửi một HTTPS request POST `/api/v1/orders/create` kèm thông tin giỏ hàng lên hệ thống.
2. **Qua Gateway:** API Gateway tiếp nhận request. Nó kiểm tra token của user (Auth), thấy hợp lệ liền chuyển tiếp request này tới **Order Service**.
3. **Xử lý đơn hàng:** **Order Service** tiếp nhận yêu cầu, tính toán tiền nong, ghi nhận một bản ghi đơn hàng mới với trạng thái **PENDING** (Chờ xử lý) vào Order Database (MongoDB).
4. **Phát tín hiệu:** Ngay sau khi ghi Database thành công, **Order Service** gửi một event tên là `order.created` chứa thông tin (Mã đơn hàng: 999, Số tiền: 500k, User: Minh) vào **Message Broker** (Kafka). Lúc này, **Order Service** lập tức trả về cho Client thông báo: "Đơn hàng của bạn đã được tiếp nhận".

Luồng đi của dữ liệu trên MSA

Ví dụ: Hành trình đặt mua một món hàng. Khách hàng bấm nút "Xác nhận đặt hàng và Thanh toán tiền mặt (COD)" trên App di động.

5. Giao tiếp bất đồng bộ hậu trường: Lúc này trên Kafka đang có tin nhắn *order.created*. Hai dịch vụ khác sẽ vào cuộc:

- **Payment Service** nghe thấy sự kiện *order.created*, nó tự động nhận tin nhắn về, tạo một yêu cầu thanh toán (hoặc hóa đơn chờ thu hộ COD) lưu vào Payment DB (MySQL). Sau đó nó gửi ngược lại event *payment.done* lên Kafka.
- **Notification Svc** nghe thấy sự kiện *order.created*, lập tức tạo một tin nhắn "Cảm ơn bạn đã đặt hàng..." rồi đẩy vào Notif DB (Redis) để gửi một Notification theo thời gian thực về điện thoại của khách hàng.

6. Cập nhật trạng thái sau cùng: **Order Service** lúc này lại nghe thấy sự kiện *payment.done* từ Payment Service gửi ra, nó hiểu là thủ tục thanh toán/COD đã chuẩn bị xong, liền cập nhật trạng thái đơn hàng trong MongoDB từ **PENDING** sang **SUCCESS** (Thành công).

So sánh MVC hiện đại và MSA

Tiêu chí so sánh	MVC hiện đại	MSA
Bản chất tổ chức	Phân tách cấu trúc bên trong một ứng dụng duy nhất.	Phân tách toàn bộ hệ thống thành nhiều ứng dụng nhỏ độc lập.
Quản lý Dữ liệu	Tập trung: Một Cơ sở dữ liệu lớn duy nhất cho toàn bộ thực thể hệ thống.	Phân tán: Mỗi dịch vụ sở hữu và toàn quyền quyết định Database riêng (SQL hoặc NoSQL).
Triển khai	Tất cả được đóng gói thành 1 file (ví dụ: app.jar hoặc docker image duy nhất) và deploy cùng lúc.	Từng dịch vụ được đóng gói và deploy độc lập bất cứ lúc nào mà không ảnh hưởng lẫn nhau.
Giao tiếp nội bộ	Gọi hàm trực tiếp trên bộ nhớ máy tính (In-memory), tốc độ tính bằng mili-giây hoặc micro-giây.	Gọi qua môi trường mạng sử dụng REST, gRPC hoặc Message Queue. Phát sinh độ trễ mạng.

So sánh MVC hiện đại và MSA

Tiêu chí so sánh	MVC hiện đại	MSA
Sự phụ thuộc công nghệ	Bị ràng buộc vào một nền tảng công nghệ chính (ví dụ: Toàn bộ hệ thống phải dùng C# .NET).	Hoàn toàn linh hoạt. Service làm AI dùng Python, Service xử lý Core dùng Java, Service hiển thị dùng Node.js.
Độ nhất quán dữ liệu	Dễ dàng kiểm soát Transaction (Ví dụ: Hủy đơn hàng và hoàn kho cùng thành công hoặc cùng thất bại).	Xử lý tính Nhất quán sau cùng: Phải sử dụng các Pattern phức tạp như Saga để đồng bộ dữ liệu giữa các DB.
Độ phức tạp vận hành	Thấp. Dễ setup CI/CD (tích hợp liên tục/triển khai liên tục), dễ tìm lỗi (Debug) bằng cách chạy ứng dụng cục bộ trên máy tính cá nhân.	Rất cao. Đòi hỏi kiến thức sâu về DevOps, Docker, Kubernetes, quản lý Log tập trung (ELK Stack) và Tracing API.

Tài liệu API và OAS/Swagger

- **API Documentation (Tài liệu API)** là hướng dẫn chi tiết giúp lập trình viên hiểu cách tích hợp và sử dụng các API. Nó bao gồm các endpoint (URL), phương thức (GET, POST,...), tham số truyền vào và cấu trúc dữ liệu trả về.
- **OpenAPI Specification (OAS)** là một chuẩn đặc tả dành cho RESTful API, sử dụng định dạng YAML hoặc JSON.
- **Swagger** là bộ công cụ (do SmartBear phát triển) để hiện thực hóa chuẩn OpenAPI, giúp thiết kế, xây dựng và tài liệu hóa API một cách trực quan.

Cấu trúc cơ bản của file YAML

- **openapi**: Phiên bản OpenAPI sử dụng.
- **info**: Thông tin chung về tài liệu
 - **title**: Tiêu đề
 - **version**: Phiên bản
- **servers**: Danh sách các Servers dành cho từng môi trường (Dev, Staging, Product)
 - **url**: Địa chỉ

```
openapi: 3.0.3
info:
  title: Sample API
  version: 1.0.0
servers:
  - url: http://api.example.com/v1
paths:
  /users:
    get:
      summary: Lấy danh sách user
```

Cấu trúc cơ bản của file YAML

- **paths:** Nơi định nghĩa các endpoint và các phương thức
 - **get**
 - **post**
 - ...
- **components:** Chứa nhiều loại thành phần cấu hình khác nhau.
 - **schemas:** Quy định các "khuôn mẫu" dữ liệu (Data Models) mà API sẽ nhận vào hoặc trả về.
 - **securitySchemes:** Các phương thức bảo mật mà API sử dụng
 - ...

```
components:
  schemas:
    User:           # Tên của schema
                    # Kiểu dữ liệu là một đối tượng
    type: object    # Các thuộc tính bên trong
    properties:
      id:
        type: integer
      username:
        type: string
```

```
components:
  securitySchemes:
    BearerAuth:     # Tên cơ chế tự đặt
    type: http
    scheme: bearer
    bearerFormat: JWT # Yêu cầu truyền Token dạng JWT để xác thực
```



CẢM ƠN CÁC BẠN ĐÃ CHÚ Ý LẮNG NGHE